YAML : block scalars (`|`, `>`, `-`) et conditions `if:` de GitHub Actions

Un workflow GitHub Actions est du YAML (YAML Ain't Markup Language), et deux difficultés se cumulent dès qu'on écrit une condition un peu longue : **comment écrire une valeur sur plusieurs lignes** (block scalars YAML) et **comment l'expression est évaluée** (syntaxe `${{ }}` et fonctions de statut de GitHub Actions). Ce sont deux couches distinctes : YAML produit une chaîne, puis GitHub Actions évalue cette chaîne.

> **Où se trouve l'exemple.** Le `if: >-` étudié ici est dans le `ci.yml` du dépôt **Java** (ci.yml:115-117, job `build`, branche `ci/semantic-release` au moment de la rédaction). Le `ci.yml` du dépôt **Angular** (ci.yml) n'a pas encore de job `release` : son `build` n'a que `needs: test` et `if: github.event_name == 'push'` (lignes 69-70). Les `run: |` multi-lignes, eux, sont bien dans les deux dépôts.

Les block scalars YAML

Un *block scalar* écrit une chaîne sur plusieurs lignes indentées. Il se règle avec deux indicateurs : le **style** puis le **chomping** (traitement des retours à la ligne finaux).

```
clé: <style><chomping>
  texte indenté
```

Le style : `|` ou `>`

Style	Nom	Effet sur les retours à la ligne
`|`	literal	conservés tels quels
`>`	folded	un retour simple devient une espace ; une ligne vide devient un vrai retour

```yaml
literal: |
  ligne 1
  ligne 2
# → "ligne 1\nligne 2\n"

folded: >
  ligne 1
  ligne 2
# → "ligne 1 ligne 2\n"

paragraphes: >
  paragraphe A
  suite de A

  paragraphe B
# → "paragraphe A suite de A\nparagraphe B\n"
```

Le chomping : rien, `-` ou `+`

Indicateur	Nom	Retours à la ligne finaux
(rien)	clip	**un seul** `\n` conservé
`-`	strip	**tous supprimés**
`+`	keep	**tous conservés**

```yaml
a: >
  texte
# → "texte\n"

b: >-
  texte
# → "texte"

c: |-
  texte
# → "texte"
```

Le `-` sert quand un `\n` parasite gênerait : une expression, un nom, une valeur comparée.

Pourquoi pas un scalar sur une seule ligne ?

Un scalar *plain* (sans guillemets) peut aussi s'étendre sur plusieurs lignes, mais il se heurte aux caractères spéciaux YAML : un `!` initial est lu comme une étiquette (*tag*), `:` et `#` changent le sens. Un block scalar `>-` n'a pas ces pièges : tout le contenu est du texte brut.

Quel style pour quel usage en CI ?

Besoin	Choix	Exemple
Script shell : une commande par ligne	`|`	`run: |`
Une seule commande longue, coupée pour la lisibilité	`>`	`run: >` + `docker build --tag ... .`
Condition ou expression longue, sans `\n` final	`>-`	`if: >-`

```yaml
# Un script : chaque ligne est une commande (| conserve les retours à la ligne)
- run: |
    docker compose up -d --wait
    curl -fsS http://localhost:8080/

# Une commande longue : > la replie sur une seule ligne
- run: >
    docker build
    --tag mon-image:latest
    .
```

Les conditions `if:` de GitHub Actions

Syntaxe `${{ }}`

Dans `if:`, les accolades sont facultatives (`if: always()` fonctionne). Elles deviennent utiles avec un bloc multi-lignes et avec un `!` initial, que YAML lirait comme un tag sans elles.

Les fonctions de statut

Fonction	Vraie quand…
`success()`	toutes les étapes ou jobs précédents ont réussi (**valeur implicite**)
`failure()`	une étape ou un job précédent a échoué
`cancelled()`	le workflow a été annulé
`always()`	toujours, même annulé

Piège central : sans fonction de statut dans le `if:`, GitHub ajoute un `success()` implicite. Un job qui dépend (`needs:`) d'un job **ignoré** (`skipped`) est alors ignoré à son tour. Dès qu'une fonction de statut apparaît dans le `if:`, l'implicite disparaît et c'est à nous de tester les résultats avec `needs.<job>.result`, qui vaut `success`, `failure`, `cancelled` ou `skipped`.

`!cancelled()` est préférable à `always()` : le job ne tourne pas si on annule le workflow à la main.

Dans DataShare

Le job `build` du dépôt Java (ci.yml:111-118) dépend de `test` et de `release`. `release` n'a lieu que sur `main` (il est `skipped` ailleurs) :

```
build:
  needs: [test, release]
  if: >-
    ${{ !cancelled() && github.event_name == 'push' && needs.test.result == 'success'
    && (needs.release.result == 'success' || needs.release.result == 'skipped') }}
```

`>-` replie l'expression sur une seule ligne, sans `\n` final. Chaque partie a un rôle :

Partie	Rôle
`!cancelled()`	désactive le `success()` implicite, sans s'exécuter si le workflow est annulé
`github.event_name == 'push'`	pas de build d'image sur une pull request (push impossible depuis un fork)
`needs.test.result == 'success'`	on ne publie jamais une image dont les tests échouent
`(… 'success' || … 'skipped')`	`release` a réussi, ou n'a pas eu lieu (hors `main`) ; son `failure` bloque le build

Les parenthèses sont indispensables : `&&` est prioritaire sur `||`.

Résultat : sur `main`, `build` attend la release ; sur une autre branche, `release` est ignoré mais `build` s'exécute si les tests passent ; si `release` échoue ou si les tests échouent, `build` est ignoré.

Les scripts multi-lignes (`run: |`) sont dans les deux dépôts, par exemple dans le `ci.yml` Angular (ci.yml:88) pour calculer le nom et les tags de l'image.

À retenir

1. Deux couches : **YAML** fabrique une chaîne (block scalar), puis **GitHub Actions** l'évalue (`${{ }}`).
2. `|` **conserve** les retours à la ligne (scripts), `>` les **replie** en espaces (commandes et expressions longues) ; le `-` **supprime le** `\n` **final**.
3. Une fonction de statut dans un `if:` (`!cancelled()`, `always()`, `failure()`) **retire le** `success()` **implicite** : sans elle, un job qui dépend d'un job `skipped` est ignoré.
4. Ce `if: >-` de `build` exprime la règle métier : tests réussis, release réussie **ou** ignorée, uniquement sur un push, jamais si le workflow est annulé.
5. `&&` est prioritaire sur `||` : toute alternative combinée à d'autres conditions se met entre parenthèses.